

CAT-IA: cómo mostrar y explicar tu proyecto

Guía práctica para abrir la aplicación, recorrer el código comentado y defender las decisiones.

Complementa docs/05_guia_defensa.md, que contiene los conceptos y las preguntas de estudio. Practica las acciones tú mismo antes de la evaluación; el segmento de dominio técnico exige resolver sin apoyo externo.

1. Prepara la demostración en este computador

1. Abre la carpeta C:\Users\anton\Desktop\evaluacion1-ia en VS Code. Usa Terminal > Nueva terminal; confirma que PowerShell esté situado en esa carpeta.
2. Abre <http://127.0.0.1:8000>. Si la aplicación ya responde, continúa con el paso 5. Evita abrir otro servidor en el mismo puerto.
3. Si está detenida, ejecuta `iniciar_modelo_local.bat` con doble clic. Este equipo tiene Ollama portable y el modelo preparado; deja la ventana abierta. Si el puerto 11435 ya está ocupado por el servicio existente, comprueba ese servicio antes de intentar iniciarlo otra vez.
4. Ejecuta `iniciar.bat` y deja su ventana abierta. Cuando indique que Uvicorn está disponible, abre <http://127.0.0.1:8000>. Para detener un servidor que tú iniciaste, usa Ctrl+C en su ventana.
5. Comprueba el aviso superior. Para demostrar inferencia real debe indicar MODO LLM y el modelo `qwen2.5:1.5b`. Si indica MODO DEMOSTRACIÓN, estás usando el simulador por reglas. El aviso refleja configuración: envía un ticket conocido para comprobar que la inferencia realmente responde.
6. En este equipo, `.env` debe seleccionar `CATIA_PROVIDER=ollama`, `CATIA_RETRIEVER=lexical` y `OLLAMA_URL=http://127.0.0.1:11435`. Edita únicamente si necesitas corregirlo; conserva los demás valores. Reinicia la aplicación después de cambiar `.env`. No muestres archivos con credenciales durante una exposición.
7. Deja abiertas tres ventanas: presentación PDF, aplicación en el navegador y VS Code. En VS Code prepara `catia/agent.py`, `catia/retrieval.py`, `catia/generation.py`, `catia/config.py`, `catia/api.py`, `prompts/v3.txt` y `tests/test_system.py`.
8. Haz un ensayo antes de la clase. En la evaluación guardada en CPU, la mediana fue cercana a 45 segundos y el percentil 95 a 71 segundos por caso. No son tiempos garantizados: reserva tiempo y explica la arquitectura mientras esperas. El botón se desactiva durante la solicitud; no recargues ni envíes consultas simultáneas.

2. Si vas a presentar desde otro computador

1. Extrae `dist/CAT-IA_entrega.zip` y abre la carpeta CAT-IA extraída. El ZIP contiene código y documentos, pero excluye `.venv`, `.env`, Ollama portable y los modelos.

2. Con Python 3.12 instalado, ejecuta `instalar.bat`. Este archivo crea el entorno virtual, instala `requirements.txt` y crea `.env` desde el ejemplo únicamente si aún no existe. La descarga de dependencias necesita internet.
3. Para practicar sin LLM, conserva `CATIA_PROVIDER=demo` y ejecuta `iniciar.bat`. Anuncia que se trata de una simulación del flujo.
4. Para usar un LLM real en ese equipo, sigue la instalación estándar indicada en `README.md` y descarga el modelo con el comando `ollama pull qwen2.5:1.5b`. Configura `.env` con `CATIA_PROVIDER=ollama` y `OLLAMA_URL=http://127.0.0.1:11434`, correspondiente al puerto estándar. Inicia Ollama y después CAT-IA.
5. No uses `iniciar_modelo_local.bat` en una extracción que no tenga `.tools/ollama`. Verifica una consulta real con anticipación; el día de la presentación no conviene depender de una descarga de modelo.

3. Apertura: qué decir en los primeros 40 segundos

«CAT-IA es un prototipo para una mesa de soporte de Nexo TI, una organización ficticia. Busca ayudar al operador a clasificar solicitudes y preparar respuestas con evidencia. Combina siete documentos internos simulados con dos síntesis de guías reales de Mozilla. Recupera fragmentos, consulta un LLM local y verifica el formato y las citas antes de mostrar un borrador. La persona conserva la decisión final».

Muestra la diapositiva del problema y luego la arquitectura. Explica que los beneficios organizacionales son objetivos por validar, no ahorros medidos en una empresa real. Relaciona el problema con requisitos concretos: clasificación, prioridad consistente, evidencia visible, abstención y trazabilidad.

4. Demostración principal: acceso web con fuentes internas y externas

1. En la aplicación, pulsa el ejemplo Acceso web. El texto describe un portal que no inicia sesión en Firefox y consulta por cookies y datos del sitio.
2. Selecciona Una persona y 4 fragmentos. Deja desmarcadas las casillas de servicio interrumpido e incidente de seguridad. En Opciones de recuperación, conserva Internas y externas.
3. Antes de enviarlo, explica: «El texto será recuperado contra el corpus. El impacto lo declara el operador y determina la prioridad por reglas; no se deduce automáticamente de la redacción».
4. Pulsa Analizar con evidencia una sola vez. Mientras responde, señala en el diagrama el trayecto: operador y API, agente, recuperación, contexto y LLM, validación y revisión humana.
5. Cuando termine, muestra categoría, prioridad y borrador. Para este ejemplo se espera Soporte Técnico. Con los campos anteriores la política determina Baja y 24 horas hábiles de primera respuesta. Si el LLM clasifica mal, identifica el error y revisa la evidencia; no lo presentes como acierto.
6. Lee una recomendación del borrador y busca su respaldo en Fuentes recuperadas. Señala una fuente interna y una externa si ambas aparecen. Explica qué aporta cada una: procedimiento organizacional y orientación técnica. La fuente externa es una síntesis local con procedencia; el sistema no navega la web en cada consulta.

7. Muestra un bloque de cita y comprueba que su oración aparece en el fragmento asociado. Di: «El identificador y el extracto se comprueban automáticamente. Yo reviso además si esa oración sostiene la recomendación». No confundas similitud con confianza o probabilidad de acierto.
8. Abre Ver recorrido del agente. Describe preparar, recuperar, consultar política, generar, validar y revisión. Este recorrido registra pasos del programa; no expone razonamiento interno del modelo.
9. Pulsa Descargar resultado JSON. Abre el archivo descargado en VS Code y localiza `trace_id`, `prompt_version`, `prompt_sha256`, `corpus_sha256`, `sources`, `decision` y `timings`. Esto permite investigar una ejecución concreta. Usa un ticket ficticio: el resultado descargado contiene más información que la traza persistida.

5. Demostración de límite: consulta sin evidencia

1. Pulsa Sin evidencia. Conserva el filtro Internas y externas y los valores de impacto del ejemplo anterior.
2. Pulsa Analizar con evidencia. La consulta astronómica no pertenece a la base de soporte; en el comportamiento verificado devuelve `insufficient_context`, sin fuentes recuperadas ni citas.
3. Explica: «El agente detecta que no recuperó evidencia y se abstiene antes de llamar al LLM. Su tarea es asistir con el corpus de soporte, no improvisar cualquier respuesta».
4. Abre `catia/agent.py` y localiza `if not hits` dentro de `run`. Relaciona esa condición con el resultado de pantalla. Muestra `test_unknown_query_abstains` en `tests/test_system.py` como evidencia automatizada del requisito.

6. Demostración opcional de prioridad y SLA

1. Usa Cobro duplicado con impacto Una persona y ambas casillas desmarcadas. Se espera categoría Facturación; la prioridad calculada es Baja, con 24 horas hábiles de primera respuesta.
2. Conserva el texto y selecciona Toda la organización junto con El servicio está interrumpido. Repite. La prioridad pasa a Alta y la primera respuesta a una hora hábil. Para un caso real el operador debe verificar que ese impacto sea verdadero; aquí es una variación didáctica.
3. Muestra `priority_for` en `catia/agent.py`: seguridad o caída de toda la organización producen Alta; caída o impacto de equipo/organización producen Media; el resto, Baja. La categoría la propone el generador, pero la prioridad sigue esta política determinista.
4. Señala que un SLA de primera respuesta no garantiza solución dentro de ese tiempo. Esta distinción está visible en la aplicación y evita promesas que el modelo no puede autorizar.

7. Recorrido del código: orden de lectura y explicación

Los archivos Python ahora tienen docstrings en clases y funciones, además de comentarios en pasos importantes. Los docstrings explican la responsabilidad; los comentarios explican criterios o límites. Lee el flujo siguiendo la entrada y la salida de cada componente, sin memorizar todas las líneas.

catia/api.py: empieza en `create_app`. El ciclo lifespan construye el agente una vez al arrancar. `POST /api/tickets/analyze` recibe un Ticket y llama a `Agent.run`. `GET /api/health` informa el modo configurado y

GET /api/sources muestra procedencia. La aplicación sirve también HTML, CSS y JavaScript. Di: «La API conecta la interfaz con el caso de uso».

catia/schemas.py: muestra Ticket y Decision. Pydantic valida tipos, longitudes y categorías. Un ticket inválido recibe HTTP 422 antes del flujo principal. Evidence relaciona un fragmento con su cita. Di: «Un JSON válido sintácticamente todavía puede ser incorrecto para nuestro contrato».

catia/config.py y config/rag.json: config.py define límites y valida; rag.json contiene los valores activos. .env selecciona proveedor, modelo y recuperación. Si cambias rag.json o .env, reinicia la aplicación. En JSON no se insertaron comentarios porque el formato no los admite; sus campos están explicados en config.py y en la sección siguiente.

catia/agent.py: recorre run desde arriba: redacta patrones sensibles, recupera, aplica prioridad, busca la política obligatoria, genera, valida citas y devuelve resultado. Si falta evidencia o falla el proveedor, se abstiene con estado explícito. La traza persistida excluye ticket y respuesta. Es un flujo acotado; no administra por sí solo sistemas de clientes.

catia/retrieval.py: load_chunks lee el manifiesto, valida rutas y hashes y construye ventanas solapadas. Retriever crea el índice TF-IDF en memoria. search transforma la consulta, calcula similitud, ordena y filtra por tipo, umbral, cantidad y presupuesto. La ruta híbrida añade embeddings, pero las métricas entregadas se obtuvieron con lexical.

catia/generation.py: build_messages separa instrucciones de sistema y datos. OllamaGenerator.generate prepara un esquema con citas permitidas, llama a /api/chat y valida Decision. DemoGenerator usa reglas y extractos, sin ejecutar el prompt en un LLM. Señala ambas clases para demostrar que sabes qué motor estás usando.

prompts/v3.txt: identifica rol, taxonomía por intención, ejemplos few-shot, límites, citas y tratamiento de datos no confiables. Explica el ejemplo de cotizar soporte: la intención es comercial aunque el producto sea técnico. No agregues comentarios de programación al prompt: cualquier texto allí forma parte de la instrucción enviada al LLM.

data/sources.json y archivos de data: muestra una entrada interna y una externa, su ID, archivo, origen, versión y hash. Relaciona doc_id con el documento y source_id con el fragmento que se cita. Un hash detecta modificación de bytes; no acredita por sí solo verdad o calidad de la fuente.

web/index.html: estructura el formulario, panel de resultado y evidencia. Los IDs permiten que JavaScript encuentre cada elemento. **web/style.css:** define colores, distribución, foco visible y adaptación móvil. **web/app.js:** gestiona ejemplos, envío JSON, estados de espera, renderizado y descarga. Utiliza textContent para mostrar respuestas como texto, sin interpretarlas como HTML.

tests/test_system.py: contiene fixtures, pruebas del corpus, prioridad, abstención, contratos y API. Los mocks reemplazan llamadas externas para aislar comportamiento. **tests/test_evaluation.py:** comprueba la separación de datos y que las métricas humanas permanezcan pendientes hasta medirse. Ninguno de esos mocks demuestra calidad del LLM.

scripts/evaluate.py: recorre casos etiquetados, guarda resultados por caso y calcula métricas.

scripts/build_documents.py: convierte documentación y evidencia existente en PDF.

scripts/package.py y check_package.py: empaquetan archivos permitidos y comprueban integridad y arranque desde extracción. La comprobación reutiliza dependencias instaladas; no equivale a probar un

sistema operativo nuevo.

Otros scripts: smoke_llm ejecuta una inferencia real; debug_generation diagnostica un caso fijo con v2; source_hashes actualiza el manifiesto después de revisar fuentes; check_browser comprueba la interfaz con un navegador preparado; check_documents audita límites de página; prepare_local_ollama es una utilidad de preparación, no parte del flujo por ticket. Los archivos __init__.py identifican paquetes Python. Los lanzadores .bat tienen comentarios REM sobre instalación, inicio y verificación.

8. Parámetros que debes saber explicar

- chunk_words = 110: cantidad máxima de palabras por fragmento. Palabras no son tokens.
- overlap_words = 20: palabras repetidas entre ventanas; el avance es 110 - 20 = 90.
- top_k = 4: máximo solicitado de fragmentos, no cantidad garantizada. La pantalla permite cambiarlo por consulta.
- min_score = 0.06: similitud mínima admitida; puede excluir evidencia débil, pero no detecta toda irrelevancia.
- max_context_chars = 6500: presupuesto del texto de fragmentos. No cuenta el prompt completo ni controla exactamente los tokens del modelo.
- semantic_weight = 0.55: peso de embeddings en modo híbrido; en lexical la señal densa no participa.
- prompt_version = v3: archivo de instrucciones seleccionado. Las versiones conservadas permiten comparar decisiones de prompting.

9. Cómo mostrar las pruebas paso a paso

1. En VS Code abre tests/test_system.py y busca test_priority_policy. Explica sus entradas parametrizadas y el resultado esperado. Busca después test_citation_rejection: altera una cita y espera su rechazo. Estos ejemplos verifican requisitos, no solo que una función devuelva algo.

2. En una terminal ubicada en la raíz del proyecto ejecuta:

```
.\venv\Scripts\python.exe -m pytest -q
```

3. Espera el resumen. La suite verificada contiene 33 casos; muestra el resultado que obtenga tu ejecución. Si aparece un fallo, lee el caso y su aserción; no lo ocultes. Un aviso de deprecación no es lo mismo que un test fallido.

4. Para actualizar la evidencia de cobertura y JUnit, ejecuta antes de regenerar los PDF:

```
.\venv\Scripts\python.exe -m pytest --cov=catia --cov-report=json:reports/coverage.json  
--junitxml=reports/tests.xml
```

5. Abre reports/RESUMEN_RESULTADOS.md y reports/OPTIMIZACION_PROMPT.md. Distingue pruebas de software, evaluación de recuperación e inferencia real. En los 20 casos sintéticos reutilizados, v3 registró macro-F1 1,00 frente a 0,78 de v2. Como v3 se ajustó después de inspeccionar esos casos, esto es regresión sobre casos conocidos, no generalización independiente.

6. Explica que precisión documental mide qué parte de lo recuperado es relevante; recall mide cuánto de lo relevante se recuperó; macro-F1 promedia el desempeño por categoría. La cobertura de líneas indica código ejecutado durante tests, no porcentaje de requisitos cumplidos o de respuestas correctas.

7. La revisión humana de fidelidad y relevancia sigue pendiente en `evaluation/revision_humana.csv`. Revisa afirmaciones contra contexto y completa resultados reales antes de atribuir puntuaciones humanas. La inspección cualitativa asistida disponible no sustituye esa revisión.

8. No ejecutes toda la evaluación Ollama de 20 casos durante una presentación breve. Muestra el informe guardado con su modelo, fecha y configuración, y una consulta real en vivo. Para repetir el experimento fuera de la exposición: `.\venv\Scripts\python.exe -m scripts.evaluate --provider ollama --prompt v3`.

10. Cambio en vivo que puedes practicar

Requerimiento de ejemplo: reducir la cantidad de contexto. Usa Acceso web con `k=4` y guarda el resultado JSON. Cambia Fragmentos a recuperar a 2, conserva texto y filtros y vuelve a analizar. Compara los IDs y documentos recuperados. Explica que menos contexto puede reducir ruido, pero perder información pertinente. No prometas que siempre mejorará la respuesta ni que llegarán exactamente `k` fragmentos.

Si te piden mostrar la implementación, abre `Ticket.top_k` en `schemas.py`, `Agent.run` donde pasa `ticket.top_k` al recuperador y `Retriever.search` donde limita hits. Para cambiar el valor global, edita `top_k` en `config/rag.json` y reinicia; recuerda que el formulario manda su propio valor y lo puede sobrescribir para esa consulta.

Requerimiento de ejemplo: cambiar el estilo del borrador. En modo LLM abre `prompts/v3.txt` y agrega una instrucción concreta compatible con el esquema, por ejemplo pedir tres pasos breves en `suggested_reply`. Guarda, repite el mismo ticket y comprueba contenido y citas. El archivo de prompt se lee en cada generación; este cambio no requiere reconstruir el índice. En modo demo no cambia la respuesta por prompting, porque ese proveedor no interpreta el prompt.

Conserva una copia del prompt antes del ejercicio y restaúrala al terminar el ensayo. Si decides entregar una versión nueva, registra el cambio y vuelve a evaluar: los informes anteriores corresponden al hash anterior del prompt. Los comentarios del código no alteran ese archivo.

11. Si algo falla durante la demostración

No abre la página: revisa la ventana de `iniciar.bat` y la dirección `127.0.0.1:8000`. Error de puerto ocupado puede indicar una instancia ya iniciada; prueba esa instancia antes de abrir otra.

La página abre, pero aparece `provider_error`: la API está disponible, pero no logró una respuesta válida del proveedor. Revisa Ollama, modelo y coincidencia de puertos con `.env`. `GET /api/health` no detecta por sí solo ese fallo. Si necesitas continuar con demo, anuncia el cambio y reinicia con `CATIA_PROVIDER=demo`; no lo atribuyas al LLM.

Aparece `invalid_output`: el contenido no pasó validación. Muestra la abstención y revisa el contrato; no inventes una respuesta para ocultar el error. **Aparece `insufficient_context`:** examina consulta, filtro y umbral antes de asumir un fallo del generador.

Cambiaste configuración y no cambia el resultado: reinicia la API y comprueba la configuración en `/api/health`. El filtro y `k` del formulario se envían en cada ticket y pueden diferir del valor global.

Error de hash al iniciar: hubo cambios en una fuente. Revisa si eran intencionales antes de ejecutar `scripts.source_hashes` y reiniciar. Actualizar el hash sin revisar elimina la señal del cambio, no demuestra que el contenido sea correcto.

12. Orden sugerido para una defensa de 12 minutos

- 0:00 a 1:00: problema, organización, usuario y alcance.
- 1:00 a 2:30: fuentes, requisitos y diagrama de arquitectura.
- 2:30 a 3:30: prompt, contratos y decisiones de recuperación.
- 3:30 a 6:30: Acceso web, evidencia, recorrido y caso sin evidencia.
- 6:30 a 8:00: código de una validación y ejecución breve de `pytest`.
- 8:00 a 9:30: resultados, límites y revisión semántica pendiente.
- 9:30 a 11:30: cambio pequeño de `k` y explicación del efecto observado.
- 11:30 a 12:00: objetivos pendientes y aplicación del diseño al problema.

Este reparto es una sugerencia, subordinada al tiempo que asigne el docente. La demostración opcional de prioridad y la lectura extensa del código son material de ensayo o de respuesta a preguntas. Ambos integrantes deben poder recorrer todo el flujo, aunque dividan la exposición.

13. Antes de entregar

Completa integrantes, sección y datos académicos donde corresponda. Confirma con el docente la aceptación del caso ficticio. Revisa los resultados que vas a declarar y practica las preguntas de `docs/05_guia_defensa.md`. Para actualizar PDF y ZIP tras cambios finales ejecuta, en este orden, los módulos `scripts.build_documents`, `scripts.package` y `scripts.check_package` con el Python de `.venv`. La presentación es `deliverables/presentacion_CAT-IA.pdf`; el paquete completo es `dist/CAT-IA_entrega.zip`. El envío a AVA y al docente lo realiza el equipo.